SA-B+-tree: Stream-Aware Splits and Leaf Hints for Append-Heavy B+-Tree Indexes

Arhaansh Jhingan
Vellore Institute of Technology, Vellore, India
arhaanshjhingan@gmail.com

***Abstract*—Many indexes receive keys in ascending order: auto-increment identifiers, timestamps, and composite keys such as (device, time) that are written by many sources at once. A textbook B+-tree handles this poorly. Every insert descends from the root, and every split leaves the left page half empty forever. Production systems patch the single-stream case. PostgreSQL caches the rightmost leaf, and InnoDB splits at the insertion point when consecutive inserts are adjacent. Both break down when several streams are interleaved or when a few records arrive late. We present the SA-B+-tree, a small, compatible modification built from three parts: a split rule gated on per-node insert runs that always cuts just after the run and leaves 10% headroom on pure appends; a 64-slot cache of recently written leaves with exact fence keys, which lets an insert skip the descent; and a hit-rate gate that switches the cache off when it stops paying. On two million 64-bit keys the SA-B+-tree needs 1.01–1.02 page reads per insert instead of 2.98 and builds an index 43–45% smaller. It inserts 2.0–2.5× faster for sequential, multi-stream and nearly sorted keys, and stays within 1% of the textbook tree on random keys. With 8 or 64 interleaved streams, or 5% late records, it beats the combination of the two existing techniques by up to 2.4× in insert time and 11–39 percentage points in page fill.**

***Index Terms*—B+-tree, index maintenance, page splits, fill factor, time-series data**

I. Introduction

The B-tree and its B+-tree variant are the default index of practically every relational and key–value storage engine [1]–[3]. Their classic analysis assumes keys arrive in random order. In that case each insert descends the tree once, and pages settle at about $\ln 2 \approx 69\%$ occupancy [4].

Real insert streams are rarely random. Surrogate primary keys come from a counter. Event tables are keyed by timestamp. Sensor and log tables are often keyed by a composite such as `(device_id, ts)`, so each device appends to its own region of the key space while thousands of devices write concurrently. For these *append-heavy* workloads the textbook B+-tree does two wasteful things:

1) **It re-descends for every insert.** The insert lands in the same leaf as the previous one, yet it pays h page accesses, where h is the tree height.
2) **It splits in the middle.** When the active leaf overflows, half of its keys move to a new right sibling. Later keys are all larger, so they go to the right page, and the left page stays 50% full for good. The index ends up twice as large as necessary, and range scans read twice as many pages.

Mature systems ship point fixes. PostgreSQL caches the rightmost leaf and inserts there directly when the key falls inside it [5], and it splits the rightmost page unevenly according to the index fillfactor [6]. InnoDB remembers the last insert position on each page and, when consecutive inserts are adjacent, splits at the insertion point instead of the middle [7]. SQLite has a fast "balance quick" path for appending to the rightmost leaf of integer-keyed tables [8]. Each fix targets *one* stream at the right edge of the index. With several interleaved streams the rightmost-leaf cache almost never hits. The insert-point heuristic is known to under-fill pages [9], and packing pages completely full backfires as soon as a few records arrive late.

This paper describes the SA-B+-tree, a mild but measurable improvement that generalises these fixes. Our contributions are:

- a **run-gated adaptive split** (Section III-B). It activates only after a node has seen a run of adjacent inserts, cuts *after* the new key so that keys of a neighbouring stream move out of the active page, and keeps a 10% headroom on pure appends;
- a **multi-slot leaf hint cache** with exact fence keys (Section III-C). It extends the rightmost-leaf fastpath to any 64 leaves, and a **hit-rate gate** disables it on workloads without locality;
- a simple **cost model** that predicts the measured page reads and fill to three digits (Section III-E);
- an **open-source implementation and evaluation** against the textbook B+-tree and against simplified models of the PostgreSQL and InnoDB heuristics, with ablations and sweeps over the number of streams and the share of late records (Section V).

II. Background

A. Splits and page fill

A B+-tree leaf holds up to C entries. Inserting into a full leaf creates a node with $C + 1$ entries, which is cut into a left node with s entries and a right node with $C + 1 - s$. The first key of the right node becomes the separator in the parent, and the split can propagate upward. The textbook choice $s = \lfloor (C + 1)/2 \rfloor$ is optimal when the next key is equally likely to land anywhere. For ascending keys it is the worst choice: the left node never receives another key, so the steady-state fill is $\lfloor (C + 1)/2 \rfloor / C \approx 50\%$.

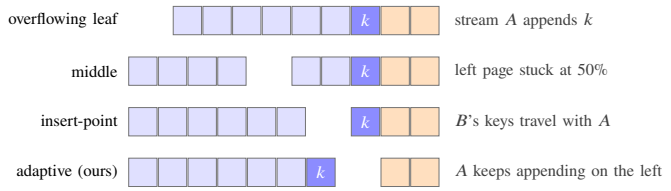


Fig. 1. Three ways to split a leaf that holds the tail of ascending stream A (blue) and the head of the neighbouring range B (orange) when A's new key k overflows it.

B. Existing heuristics

Rightmost-leaf fastpath (PostgreSQL). Since 2018, PostgreSQL's nbtree remembers the block it last inserted into if that block is the rightmost leaf. The next insert checks that the block is still the rightmost leaf, has free space, and that the key is greater than the page's first key. If so, it inserts without descending [5]. A split of the rightmost page leaves it filled to the fillfactor, 90% by default for B-tree indexes [6].

Insert-point split (InnoDB). InnoDB records the position of the last insert on each page. If the next insert lands immediately next to it, InnoDB assumes a sequential pattern and splits at the insertion point, so that the new record starts the new page [7]. The heuristic fires on a single adjacent pair. When the insert point lies in front of keys belonging to another key range, the next page inherits those keys, and pages can be drastically under-filled [9].

Fence keys. Many B⁺-tree implementations store, in each node, the interval of keys it is responsible for: a *high key* as in Lehman and Yao [10], or both low and high *fence keys* [3]. We use leaf fences to validate hints.

III. Design

The SA-B⁺-tree is an ordinary B⁺-tree: same node layout, search, scan and separator semantics. It differs only in where nodes are split and in how an insert finds its leaf. Both changes are local to the insert path.

A. Insert-direction tracking

Every node keeps two small counters, in the spirit of InnoDB's page header fields: `last_pos`, the slot of the most recent insert, and `run`, the number of consecutive inserts that landed at `last_pos`+1, i.e. immediately to the right of the previous one. Inner nodes track the slots at which separators are inserted in the same way. An erase resets the counters. After a split, the half that received the new key inherits the run.

B. Run-gated adaptive split

Let the overflowing node hold $n = C + 1$ keys, with the new key at slot p. If the node's run is shorter than a threshold R (we use $R = 3$), it splits in the middle as usual. A random insert lands right after the previous one with probability about $1/C$, so three in a row virtually never happen by chance. Otherwise the left node keeps s keys:

$$s = \begin{cases} p+1 & \text{if } p+1 < n \quad \text{(keys follow the run)} \\ \lfloor f\,C \rfloor & \text{if } p+1 = n \quad \text{(pure append)} \end{cases} \tag{1}$$

The value is clamped so both halves are non-empty; an inner node needs one key on each side of the key it moves up. Fig. 1 contrasts the rule with the alternatives.

Cutting after the new key matters when the run sits in front of someone else's keys, which is the normal state of a composite-key index with many streams. The keys after k belong to the next range and will never receive another insert from this stream. Moving them out, rather than moving k out with them as the insert-point heuristic does, leaves the stream appending to the end of a page that contains only its own keys. The next overflow of that page is a pure append.

The fill limit f applies only to pure appends. With $f = 1$ the left page is packed completely, but a single late record that belongs in it then forces a middle split, leaving two half-empty pages that never fill up again. We use $f = 0.9$, the same headroom PostgreSQL's default B-tree fillfactor leaves [6]. Section V shows that 1% late records are enough to make $f = 1$ lose.

The same rule is applied to inner nodes, using the slot at which a separator is inserted. For a single stream this packs the inner levels as well.

Algorithm 1 Insert with leaf hints (single-threaded)

```
1:  procedure Insert(k, v)
2:      if cache enabled or sampled insert then
3:          i ← last-used slot if it contains k, else scan slots
4:          if i ≠ ⊥ and leaf L_i is not full then
5:              insert (k, v) into L_i                ▷ 1 page access
6:              return
7:          end if
8:      end if
9:      descend from the root to leaf L, recording the path
10:     insert (k, v) into L; split by Eq. (1) if it overflows
11:     propagate separators up the recorded path
12:     cache the leaf now holding k in slot i (or the LRU slot)
13: end procedure
```

C. Leaf hint cache

Every leaf stores fence keys $[low, high)$, set when the leaf is created by a split and never changed afterwards: the SA-B⁺-tree never merges nodes. Fences of different leaves are disjoint and cover the key space. A small cache of K slots ($K = 64$ by default) remembers recently written leaves together with their fences. Each slot is stored as a page id, *low*, and $span = high - 1 - low$ in a structure-of-arrays layout, so that testing a slot is a single unsigned comparison $key - low \le span$.

Algorithm 1 shows the insert path. A hit costs one page access instead of h. If the cached leaf is full, the insert must split it, which requires the parent path, so it falls back to a

normal descent. That leaf then shares its slot with whichever half now holds the key: the stream's next leaf replaces the one it just filled, so one stream occupies one slot. Whenever a leaf splits, any slot caching it gets the new, smaller *high*. Cached fences are therefore always exact, and a hit can never put a key in the wrong leaf. Our invariant checker verifies this after every test batch.

The PostgreSQL fastpath is the special case $K = 1$ restricted to the rightmost leaf. With K slots, K concurrent streams each keep their active leaf cached.

D. Hit-rate gating

Probing costs something. With random keys almost every probe misses, and with more streams than slots the LRU policy thrashes. Every 64 probes the cache compares its hit count with half the window. Below that it switches to *sampling*: only every 64th insert probes and refreshes it. One window of sampled hits switches it back on. Random workloads therefore probe on about 2.4% of inserts in our runs, and a workload that turns sequential regains the cache after a few thousand inserts.

E. Cost model

Consider one append stream with hints enabled, $f < 1$ and tree height h. After an append split, the new right leaf holds $C + 1 - \lfloor fC \rfloor$ entries and absorbs $\lfloor fC \rfloor$ further inserts before it overflows. All of them are hits except the last, which reads the full leaf and then descends. Hence

$$\text{reads per insert} = 1 + \frac{h}{\lfloor fC \rfloor}, \qquad \text{leaf fill} = \frac{\lfloor fC \rfloor}{C}. \quad (2)$$

For $C = 256$, $f = 0.9$, $h = 3$ this predicts 1.0130 reads and 89.84% fill. We measure 1.0129 (the tree is shallower early in the load) and 89.84%. For $f = 1$ it predicts 1.0117 and 100%; we measure 1.0116 and 99.99%. The textbook tree pays h reads and reaches 50% fill.

IV. Implementation

The SA-B⁺-tree is a header-only C++17 library of about 700 lines. Nodes are fixed-capacity pages (capacity reserved at allocation, so filling a page never reallocates) held in a page table indexed by page id. Every node visited by an operation counts as one logical page read, the standard cost model for disk-resident indexes. Created or modified nodes count as page writes. Leaves are chained for range scans. Erase removes the key from its leaf without rebalancing, as PostgreSQL does until VACUUM, which keeps fences immutable.

The split policy (Middle, InsertPoint, Adaptive) and the hint policy (None, Rightmost, Multi) are independent options, so the same code base provides the textbook baseline, models of the PostgreSQL and InnoDB heuristics, their combination, and every ablation. Rightmost caches only the rightmost leaf, as PostgreSQL does. InsertPoint starts the new page with the new key whenever the previous insert was adjacent (run ≥ 1). This is a simplified model of InnoDB's heuristic: InnoDB's actual split code differs in details, so this variant shows how the heuristic behaves, not how InnoDB itself performs.

Correctness is checked by differential tests that drive ten configurations, with node capacities from 3 to 256, through the same streams of inserts, overwrites, erases, lookups and range scans as a `std::map`. After each batch the tests verify key order, fence keys, uniform leaf depth, the leaf chain, and that every cached hint still matches its leaf. The suite also passes under AddressSanitizer and UBSan.

V. Evaluation

A. Setup

All experiments insert $N = 2 \times 10^6$ distinct 64-bit keys into an empty tree with $C = 256$ (about a 4 KiB page of 8-byte keys and 8-byte values), in memory, on one thread. The machine is an Intel Core Ultra 9 285H laptop running Linux 7.2, with GCC 16 at `-O3` and the benchmark pinned to a performance core. The five key orders are generated deterministically: *sequential* $(0, 1, 2, \ldots)$; *random* (a uniform permutation); *streams-K* (K ascending sequences in disjoint key ranges, interleaved uniformly at random, like `(device, ts)` keys); and *nearly sorted* (ascending, but 5% of the records are delayed by 1–1000 positions).

Page reads, writes, splits and fill are exact. Insert time is noisy on a laptop: background load and clock changes only ever add time. Each configuration is therefore built once untimed and then 7 times, interleaving the variants so that slow drift hits all of them alike, and we report the fastest run. Across complete re-runs of the suite, absolute times moved by up to about 10%, while the comparisons within one workload were stable.

We compare the textbook tree (*baseline*), the PostgreSQL-style cache (*rightmost*), the InnoDB-style split (*insert-point*), their combination (*existing*), our two components alone (*adaptive split*, *multi-hint*), and the full SA-B⁺-tree ($R = 3$, $f = 0.9$, $K = 64$).

B. Overall results

Table I and Fig. 2 summarise the main experiment. On every workload with key locality, the SA-B⁺-tree needs 1.01–1.02 page reads per insert instead of 2.98. It builds an index 43–45% smaller than the baseline, roughly halves the number of leaf splits (8,695 instead of 15,623 for sequential keys), and inserts 2.0–2.5× faster. Smaller indexes also make range scans cheaper: a scan returning 1,000 consecutive entries reads 7.3–7.4 pages instead of 10.5–10.8. Point lookups cost the same three page reads in every variant, since all trees have height 3.

On a single clean append stream the existing combination is slightly better (8.6 vs. 9.4 ns, 30.6 vs. 34.1 MiB). It packs pages to 100%, which is exactly what our $f = 1$ ablation does too (8.7 ns, 30.6 MiB). The 10% headroom is a deliberate trade, justified by the next two experiments. Everywhere else the existing techniques lose most of their benefit. With 8 or 64 interleaved streams the rightmost cache hits almost never, so inserts still cost 2.97–2.98 reads and the SA-B⁺-tree is 2.4× faster. The insert-point split reaches only 74–79% fill. With 5% late records the cache still works (1.10 reads), but fill collapses

TABLE I

TWO MILLION INSERTS, $C = 256$. TIMES ARE BEST OF 7; ALL OTHER COLUMNS ARE EXACT. *EXISTING* = RIGHTMOST-LEAF CACHE + INSERT-POINT SPLIT.

	insert time (ns)			page reads / insert			leaf fill (%)			index size (MiB)		
workload	base	exist.	**SA**	base	exist.	**SA**	base	exist.	**SA**	base	exist.	**SA**
sequential	23.6	8.6	**9.4**	2.98	1.01	**1.01**	50.0	100.0	**89.8**	61.5	30.6	**34.1**
random	222.5	219.8	**223.6**	2.98	2.98	**2.97**	70.2	70.0	**70.2**	43.8	43.9	**43.8**
8 streams	43.4	45.0	**18.4**	2.98	2.97	**1.01**	50.0	79.0	**89.8**	61.5	38.8	**34.2**
64 streams	59.2	63.9	**26.8**	2.98	2.98	**1.01**	50.1	74.4	**89.1**	61.2	41.3	**34.5**
nearly sorted	26.0	14.1	**13.1**	2.98	1.10	**1.02**	52.1	52.3	**91.5**	59.1	58.8	**33.5**

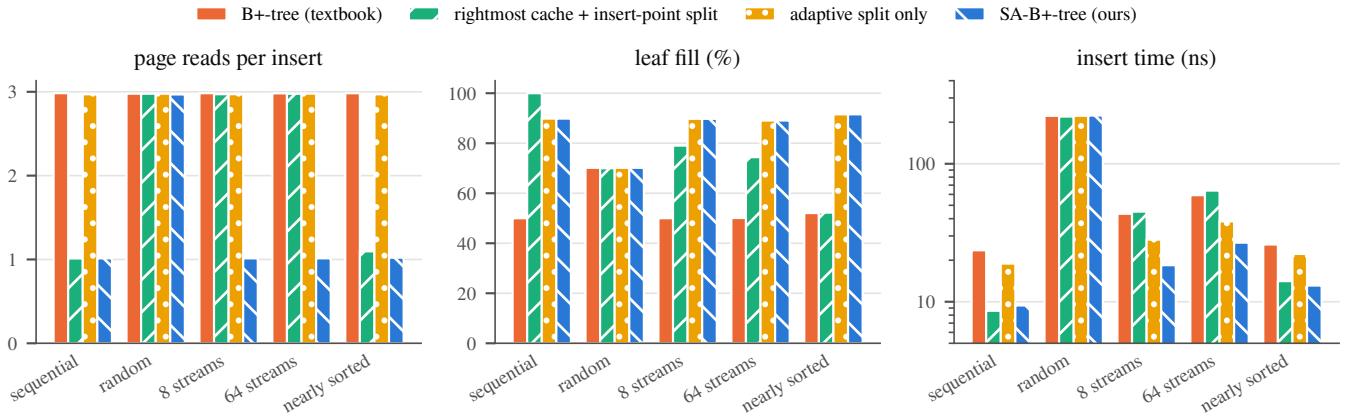


Fig. 2. Page reads per insert, leaf fill and insert time (log scale) for each workload.

TABLE II

ABLATION: CONTRIBUTION OF EACH COMPONENT (INSERT NS / PAGE READS PER INSERT / LEAF FILL %).

variant	sequential	64 streams	nearly sorted
baseline	23.6/2.98/50	59.2/2.98/50	26.0/2.98/52
rightmost	11.8/1.02/50	60.0/2.98/50	14.2/1.10/52
insert-point	18.0/2.97/100	62.9/2.98/74	25.1/2.98/52
adaptive split	18.9/2.97/90	38.2/2.98/89	22.2/2.97/92
multi-hint	11.5/1.02/50	48.9/1.02/50	14.5/1.04/52
SA-B⁺-tree	**9.4/1.01/90**	**26.8/1.01/89**	**13.1/1.02/92**
SA-B⁺-tree, f=1	8.7/1.01/100	26.6/1.01/99	14.8/1.03/52

to 52%. On random keys all variants are within the noise. A dedicated A/B run with 21 interleaved repetitions measured 224.5 ns for the SA-B⁺-tree against 222.4 ns for the baseline, a difference of 0.9%.

C. Ablation

Table II separates the two components. The adaptive split alone fixes page fill on every workload and, through fewer splits and better cache locality, speeds up multi-stream inserts by 1.5×. It does not reduce page reads. The multi-slot hint cache alone brings reads down to 1.02–1.04 but leaves pages half full. The two are complementary: the combination is faster than either part on all three workloads.

D. Number of streams

Fig. 3 varies the number of interleaved streams from 1 to 1,024. The hint cache serves 99.6% of inserts as long as the number of streams does not exceed its slots: up to 16 streams with $K = 16$, up to 64 with $K = 64$. Beyond that the gate switches it off and page reads return to the baseline. The rightmost-leaf cache already fails at two streams. The adaptive split keeps leaves 87–90% full up to 256 streams and 78% at 1,024. The insert-point heuristic drops to 67–88% (74% at 64 streams), the under-filling described in MySQL bug #67718. Even past the cache's capacity (128–1,024 streams), the SA-B⁺-tree remains 1.5× faster than the baseline thanks to the split rule alone.

E. Late records

Fig. 4 delays a growing share of records by up to 1,000 positions. Any scheme that packs pages to 100% (the existing combination and our $f = 1$ ablation) falls from 100% to 53% fill with only 1% late records. Each straggler that lands in a full page splits it in the middle, into two halves that are never filled again. With $f = 0.9$ the SA-B⁺-tree holds 90–92% fill up to 5% late records and 84% at 10%. At 20% the headroom is no longer enough: fill drops to 55%, below the baseline's 59%. The page-read saving survives (1.04 reads per insert), but the space saving does not.

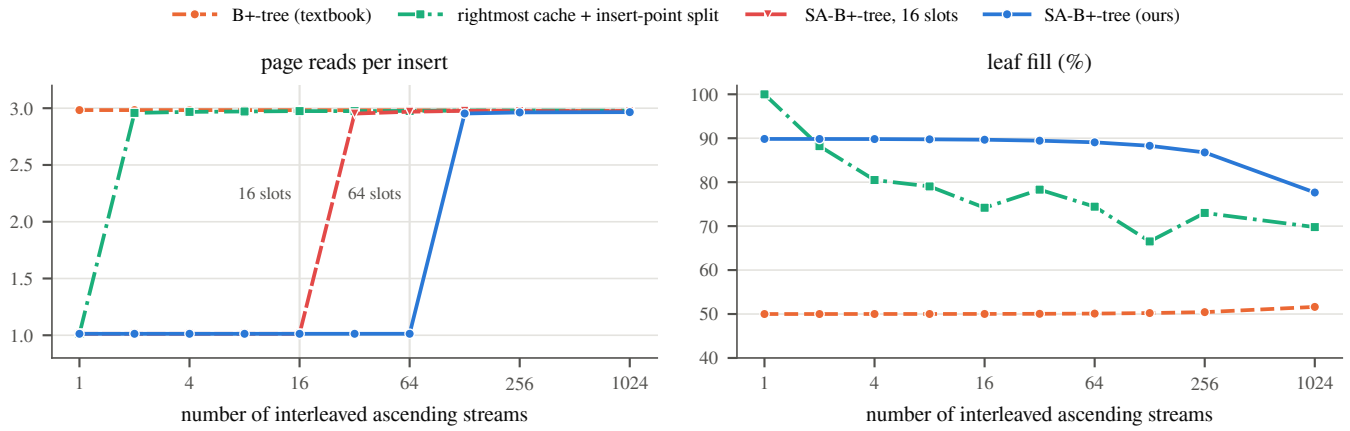


Fig. 3. Sweep over the number of interleaved ascending streams. The SA-B⁺-tree with 64 and with 16 hint slots; fill is identical for both and for the adaptive split alone, so only one curve is drawn on the right.

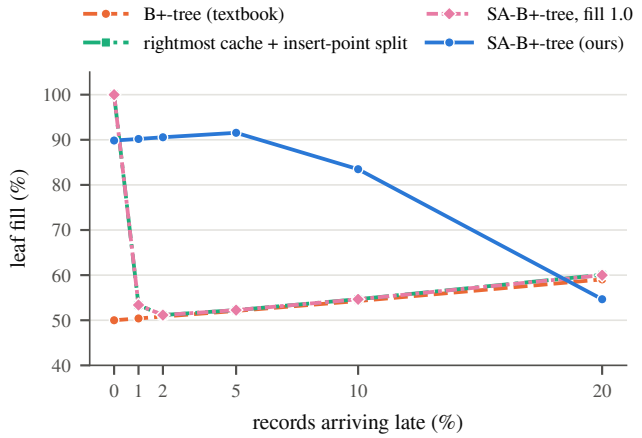


Fig. 4. Leaf fill as a growing share of records arrives up to 1,000 positions late. The curves for $f = 1$ and for the existing combination coincide.

VI. Related Work

B-trees and their maintenance are surveyed by Comer [2] and Graefe [3]. Yao's analysis gives the $\ln 2$ fill of random insertions [4]. B*-trees [11] raise fill by redistributing keys to siblings before splitting, which helps random inserts but does not remove the descent cost of appends. Lehman and Yao introduced per-node high keys for concurrent traversal [10]. We reuse the same idea, as fence keys, to validate cached leaves. Bulk loading builds packed trees from sorted input, but only when the whole input is available up front. Write-optimised structures such as the LSM-tree [12] change the storage layout altogether. The SA-B⁺-tree instead keeps the B⁺-tree layout and on-disk format unchanged, so it could replace the insert path of an existing engine. The closest work is the engineering in PostgreSQL, InnoDB and SQLite discussed in Section II, which the SA-B⁺-tree generalises from one stream at the right edge to many streams anywhere in the key space.

VII. Limitations and Future Work

The evaluation is in memory and single-threaded. In a real engine a skipped descent also saves latch acquisitions and buffer-pool lookups, so our timings are probably conservative, but we have not measured that. A concurrent version would validate a hinted leaf's fences after latching it, the same check PostgreSQL performs on its cached block. Only ascending runs are detected; descending runs need a separator chosen just above the left half's largest key and are left for future work. Nodes are never merged, which keeps fences immutable outside splits. Supporting merges would require invalidating the hints of merged pages. At 20% late records the fixed 10% headroom stops working. A natural extension is to stop applying the fill limit to a node once it observes many out-of-order inserts. Finally, separators of different streams interleave inside inner nodes, so runs there are short. At 64 streams inner-node fill is 56% instead of the baseline's 83%, which has no practical effect because inner nodes are under 1% of the pages.

VIII. Conclusion

Two inexpensive changes to the insert path of a B⁺-tree recover most of the waste of append-heavy workloads. A split rule gated on observed insert runs, which cuts after the run and keeps a little headroom, keeps pages about 90% full. A small, self-disabling cache of leaves with exact fences removes the root-to-leaf descent. Together they cut page reads per insert from about 3 to 1.01, shrink the index by 43–45% and speed up inserts 2.0–2.5×. Unlike the single-stream heuristics in production systems, they keep working with dozens of interleaved streams and with a few percent of late records, and they cost about 1% on random keys. The code, benchmarks and raw results are available at https://github.com/XArhaanshX/database-project.

References

[1] R. Bayer and E. M. McCreight, "Organization and maintenance of large ordered indexes," *Acta Informatica*, vol. 1, no. 3, pp. 173–189, 1972.

[2] D. Comer, “The ubiquitous B-tree,” *ACM Computing Surveys*, vol. 11, no. 2, pp. 121–137, 1979.
[3] G. Graefe, “Modern B-tree techniques,” *Foundations and Trends in Databases*, vol. 3, no. 4, pp. 203–402, 2011.
[4] A. C.-C. Yao, “On random 2–3 trees,” *Acta Informatica*, vol. 9, no. 2, pp. 159–170, 1978.
[5] PostgreSQL Global Development Group, “Commit 2b272734: Optimize btree insertions for common case of increasing values,” https://postgr.es/m/E1f0Qx3-0004Z5-KN@gemulon.postgresql.org, 2018.
[6] ——, “PostgreSQL documentation: CREATE INDEX, storage parameter fillfactor,” https://www.postgresql.org/docs/current/sql-createindex.html.
[7] Percona, “InnoDB page merging and page splitting,” Percona blog, https://www.percona.com/blog/innodb-page-merging-and-page-splitting/, 2017.
[8] SQLite, “SQLite source, `btree.c`: `balance_quick()`,” https://sqlite.org/src/file/src/btree.c.
[9] MySQL Bug System, “Bug #67718: InnoDB drastically under-fills pages in certain conditions,” https://bugs.mysql.com/bug.php?id=67718.
[10] P. L. Lehman and S. B. Yao, “Efficient locking for concurrent operations on B-trees,” *ACM Transactions on Database Systems*, vol. 6, no. 4, pp. 650–670, 1981.
[11] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
[12] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.